

DAY-1-SB

Wednesday, 16 September 2026 11:58 AM

1. Why was spring created?

Spring helps us achieve loose coupling

Spring Container

↓

Creates Objects

↓

Manages Objects

↓

Injects Objects

Developers focus on business logic. Spring handles object creation and wiring.

Spring is a lightweight Java framework that provides features such as Dependency Injection and Inversion of Control to build loosely coupled, maintainable, and testable applications.

Today's Key Takeaway

Before Spring:
Developer

↓

Creates Objects

↓

Connects Objects
After Spring:
Developer

↓

Defines Classes

↓

Spring Creates Objects

↓

Spring Connects Objects
This is the biggest idea in Spring.

IoC (Inversion of control):

Inversion of Control (IoC) is a design principle in which the responsibility of creating and managing objects is transferred from the application to the Spring Container.

Without IoC

```
Java
EmployeeRepository repo = new EmployeeRepository();

EmployeeService service =
    new EmployeeService(repo);
```

You control object creation.

With IoC

```
Java
@Service
public class EmployeeService {

    private final EmployeeRepository repository;

    public EmployeeService(EmployeeRepository repository) {
        this.repository = repository;
    }
}
```

Spring controls the creation and management of these objects.

```
Plain text
You
↓
Define classes

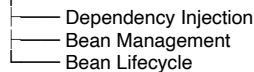
Spring Container
↓
```

Creates & manages objects

Inversion: Reverse the control of object creation.

IoC is not same as DI [dependency injection]. **Dependency Injection (DI)** is one way Spring implements IoC.

IoC



Note: **The Spring IoC Container creates, manages, configures, and injects objects (beans).**

Dependency Injection:

It's a technique where an object's required dependency is provided to it outside instead of object creating dependency itself

Type of dependency Injection:

1. Constructor injection: dependency is provided through constructor

```

</> Java
@Service
public class EmployeeService {

    private final EmployeeRepository repository;

    public EmployeeService(EmployeeRepository repository) {
        this.repository = repository;
    }

}
  
```

Here:

- `EmployeeRepository` → dependency
- `EmployeeService` → class that needs the dependency
- `EmployeeService(EmployeeRepository repository)` → constructor
- Spring provides `EmployeeRepository` through the constructor.

Conceptually, Spring does:

```

</> Java

EmployeeRepository repository = new EmployeeRepository();

EmployeeService service =
    new EmployeeService(repository);
  
```

You don't write this wiring yourself; Spring handles it.

2. Setter injection: Here, spring provides dependency through setter object once the object is created

```

</> Java
@Service
public class EmployeeService {

    private EmployeeRepository repository;

    @Autowired
    public void setRepository(EmployeeRepository repository) {
        this.repository = repository;
    }

}
  
```

The important difference is:

Constructor Injection

```

</> Plain text

Create object + provide dependency
↓
EmployeeService(repository)
  
```

Setter Injection

```

</> Plain text

Create object
↓
Provide dependency through setter
↓
setRepository(repository)
  
```

- Difference between Constructor injection & setter injection is, In constructor injection dependency is provided while creating object, but in setter injection dependency is provided once the object is created

Constructor Injection:

```

</> Java

EmployeeService service =
    new EmployeeService(repository);
  
```

The dependency is given **while creating the object**.

Setter Injection:

```
</> Java

EmployeeService service =
    new EmployeeService();

service.setRepository(repository);
```

The object is created **first**, and the dependency is provided **afterward**.

```
</> Java

@Repository
public class EmployeeRepository {

    public void save() {
        System.out.println("Employee Saved");
    }
}
```

```
</> Java

@Service
public class EmployeeService {

    private EmployeeRepository repository;

    @Autowired
    public void setRepository(EmployeeRepository repository) {
        this.repository = repository;
    }

    public void saveEmployee() {
        repository.save();
    }
}
```

Spring essentially handles this for us:

```
</> Java

EmployeeRepository repository = new EmployeeRepository();

EmployeeService service = new EmployeeService();

service.setRepository(repository);
```

You don't manually write these lines when using Spring.

- In Setter Injection, the dependency is assigned after the object is created through a setter method, so the field cannot be final. In Constructor Injection, the dependency is assigned during object creation through the constructor, so we can make the field final and ensure it cannot be reassigned later.
- In mandatory dependency, we use constructor injection because class cannot be created If a dependency is mandatory, the class should not be created without that dependency. Constructor Injection ensures this because the dependency must be provided at the time of object creation.

For example:

```
</> Java

public EmployeeService(EmployeeRepository repository) {
    this.repository = repository;
}
```

When creating the object, we **must provide** an `EmployeeRepository`:

```
</> Java

EmployeeRepository repository = new EmployeeRepository();

EmployeeService service =
    new EmployeeService(repository);
```

If we don't provide the repository, we cannot create the `EmployeeService` object using this constructor.

- In case of optional dependency, we use setter injection. the object can be created first and the dependency can be provided later through the setter method.

```
</> Java

EmployeeService service = new EmployeeService();

service.setRepository(repository);
```

So there is a possibility that the `EmployeeService` object exists **without its required dependency** before `setRepository()` is called.

Constructor Injection → dependency is required at object creation.
Setter Injection → dependency can be provided after object creation.

3. Field Injection:

Field Injection is a type of Dependency Injection where Spring directly injects the dependency into a **class field** using **@Autowired**. Instead of passing the dependency through a constructor or setter, Spring directly puts the dependency into the variable.

```
</> Java

@Service
public class EmployeeService {

    @Autowired
    private EmployeeRepository repository;

}
```

Spring sees **@Autowired** and injects the **EmployeeRepository** object into **repository**.

NOTE: Although Field Injection is simple and requires less code, **Constructor Injection is generally preferred**, especially for mandatory dependencies, because it makes dependencies explicit and allows fields to be final.

Difference between Dependency Injection & IOC

```
</> Java

public EmployeeService(EmployeeRepository repository) {
    this.repository = repository;
}
```

When creating the object, we must provide an **EmployeeRepository**:

```
</> Java

EmployeeRepository repository = new EmployeeRepository();

EmployeeService service =
    new EmployeeService(repository);
```

If we don't provide the repository, we cannot create the **EmployeeService** object using this constructor.

1. IOC is a principal, DI is a way/ technique to achieve IOC
2. IoC is a principle where control of object creation and management is transferred to the Spring Container. Dependency Injection is one technique through which Spring implements IoC by providing the required dependencies to a class instead of the class creating them itself.

IoC

Design principle

Broader concept

Says **who controls object creation**

Spring Container takes control

Can be achieved in different ways

DI

Technique/pattern

More specific

Says **how dependencies are provided**

Spring injects dependencies

Constructor, Setter, Field injection

SPRING CORE CONCEPTS:

1. Spring Bean:

A **Spring Bean** is simply an **object that is created, managed, and maintained by the Spring Container**.

The key phrase to remember:

Object managed by spring = spring bean

```
@Repository
public class EmployeeRepository {

    public void save() {
        System.out.println("Employee Saved");
    }

}
```

As we have **@repository**, spring detects **EmployeeRepository** and creates object of it and manages it as bean

@Repository → Spring Bean

@Service → Spring Bean

@Controller → Spring Bean

@Component → Spring Bean

These annotations, will tell spring to create and manage object of this class

HOW DOES SPRING CREATE BEAN?

Spring creates bean through 2 ways:

- **Component scanning** — using @Component, @Service, @Repository, @Controller
- **Java configuration** — using @Bean

1. Component scanning: Component Scanning is the process by which Spring searches your application classes for specific Spring annotations and automatically creates and manages objects (Beans) for those classes.

Without spring: You have to create object, you have to connect object and you have to manage object. Example snippet below

1. First, let's see the problem without Spring

Suppose you have three classes:

```
</> Java
public class EmployeeRepository {

    public void save() {
        System.out.println("Employee Saved");
    }
}
```

```
</> Java
public class EmployeeService {

    private EmployeeRepository repository;

    public EmployeeService(EmployeeRepository repository) {
        this.repository = repository;
    }

    public void saveEmployee() {
        repository.save();
    }
}
```

And:

```
</> Java
public class EmployeeController {

    private EmployeeService service;

    public EmployeeController(EmployeeService service) {
        this.service = service;
    }
}
```

Without Spring, you have to create all these objects yourself:

```
</> Java
EmployeeRepository repository =
    new EmployeeRepository();

EmployeeService service =
    new EmployeeService(repository);

EmployeeController controller =
    new EmployeeController(service);
```

How spring scans?

when your application starts it looks through specific set of package in your application.

```
Start Spring Boot
↓
Find the main application package
↓
Scan classes inside that package/subpackages
↓
Look for Spring component annotations
↓
Find @Controller
Find @Service
Find @Repository
Find @Component
↓
Create objects
↓
Register them as Beans
↓
Manage them
```

Spring looks for classes marked with annotations : Component, Service, Repository, Controller, Restcontroller

NOTE: Component scanning does NOT scan every class and turn every class into a Bean.

public class Employee {
}
-> here there is no component annotation, Spring doesn't automatically treat this class as a component just because it exists.

@Component
public class Employee {
}
-> Now Spring can discover it during component scanning and create a Bean.

1. **@Component**: It is generic purpose, spring managed class. Now Spring can discover it during component scanning and create a Bean.

```
@Component
public class EmailValidator {

    public boolean isValid(String email) {
        return email.contains("@");
    }
}
```

2. **@Service**: It handles business logic

```
@Service
public class EmployeeService {

    public void createEmployee() {
        System.out.println("Creating employee...");
    }
}
```

3. **@Repository** : Its for database access

```
@Repository
public class EmployeeRepository {

    public void saveEmployee() {
        System.out.println("Saving employee to database...");
    }
}
```

4. **@Controller**: It handles web request in spring MVC

```
@Controller
public class EmployeeController {

    @GetMapping("/employees")
    public String employees() {
        return "employees";
    }
}
```

5. **Java Configuration**: We use **@Bean**. It will tell **Create and manage the object returned by this method as a Spring Bean**.

```
@Configuration
public class AppConfig {

    @Bean
    public EmployeeService employeeService() {
        return new EmployeeService();
    }
}
```

Here, we let spring know, object returns from this method has to be registered as bean -> **@Bean** public EmployeeService employeeService() {
Here, we explicitly return new object -> return new EmployeeService();

NOTE:

@Component → class-level
@Bean → method-level

Using **@Component**, we tell spring to discover its class , create & manage its objects.

```
@Component
public class EmailService {
}
```

Using @Bean, we tell spring using this method to create object and manage the returned object

```
@Configuration
public class AppConfig {

    @Bean
    public EmailService emailService() {
        return new EmailService();
    }
}
```

We need @Bean, when we are using third party class where we cannot modify and add that class, because we don't own its source code

```
@Configuration
public class AppConfig {

    @Bean
    public PaymentClient paymentClient() {
        return new PaymentClient("my-api-key");
    }
}
```

Bean Scope:

Bean scope defines the lifecycle and number of instances of a Spring Bean managed by the Spring container.

Example:

@Service

```
public class EmployeeService {
```

} -> **Should Spring create one EmployeeService object or a new one every time it is requested? This is what bean scope controls**

By default bean scope will be singleton - Spring creates one instance of bean per IOC container

Other bean scopes:

Scope	Meaning
singleton	One instance per Spring container - this will have shared object as we will have only one object
prototype	New instance whenever requested - this will not have shared object
request	One instance per HTTP request
session	One instance per HTTP session
application	One instance per ServletContext
websocket	One instance per WebSocket session

Syntax to define different scopes of bean

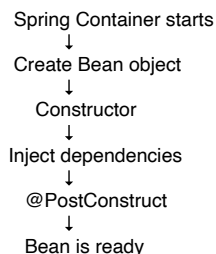
```
@Service
@Scope("prototype")
public class EmployeeService {

    public EmployeeService() {
        System.out.println("EmployeeService object created");
    }
}
```

"Singleton is the default Spring Bean scope, where one instance is created per Spring Container. Prototype scope creates a new Bean instance each time the Bean is requested from the container."

BEAN LIFECYCLE:

The Bean Lifecycle is the sequence of steps a Spring Bean goes through from the time Spring creates the Bean until Spring destroys it.





After the Bean is created

For example:

- Load configuration
- Open a connection
- Initialize resources
- Perform setup

And sometimes you need code:

Before the Bean is destroyed

For example:

- Close a connection
- Release resources
- Cleanup

So Spring provides lifecycle hooks that allow us to run code during these stages.

@Service

```

public class EmployeeService {

    public EmployeeService() {
        System.out.println("Constructor called"); -> Spring creates the object, so the constructor runs.
    }

    @PostConstruct
    public void init() {
        System.out.println("Initialization"); -> This method runs after Spring has created the Bean and injected its dependencies,
        before the Bean is ready for normal use.
    }

    @PreDestroy
    public void destroy() {
        System.out.println("Destroying Bean"); -> This method runs when Spring is destroying the Bean, typically during application
        shutdown.
    }
}
  
```

@Post construct : Runs after Bean creation and dependency injection. Used for initialization

@Pre destroy: Runs before Spring destroys the Bean. Used for cleanup

NOTE: Also, for @PreDestroy, remember that Spring manages destruction of normal singleton Beans; **prototype Beans are a special case** because Spring does not manage their full destruction lifecycle.

"The Spring Bean lifecycle starts when the Spring Container creates the Bean. Spring then injects its dependencies and calls the initialization callback, such as @PostConstruct. The Bean is then available for use. When the container shuts down, Spring calls the destruction callback, such as @PreDestroy, before destroying the Bean."

BEAN FACTORY:

Bean factory is a spring IOC container interface, which creates and manages the bean

```

@Component
public class EmployeeService {

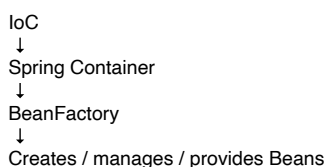
    public void hello() {
        System.out.println("Hello Employee");
    }
}
  
```

```

EmployeeService service =
    beanFactory.getBean(EmployeeService.class);

service.hello();
  
```

We are **not creating the object ourselves** with new EmployeeService(). We're asking the Spring container for the object it manages.



BeanFactory	ApplicationContext
Basic IoC container	Advanced IoC container
Basic Bean management	Bean management + many additional features
More lightweight/basic	More commonly used
Core Spring	Used extensively in Spring Boot

Application context:

ApplicationContext is an **advanced Spring IoC container** that is responsible for creating, managing, and providing Spring Beans, along with additional features such as events, internationalization, and resource handling.

You can access the ApplicationContext like this:

```
</> Java  
  
ApplicationContext context =  
    new AnnotationConfigApplicationContext(AppConfig.class);
```

Then you can retrieve a Bean:

```
</> Java  
  
EmployeeService service =  
    context.getBean(EmployeeService.class);
```

Bean factory is lazy initialization, which means The Bean may be created when you actually request it. But by default singleton Beans are generally created during application-context